1. GRAPHMIND: USER-CENTRIC LONG-TERM MEMORY MINDMAP + HYBRID RAG ASSISTANT

Theme

Graph-based memory systems for personalized AI assistants (Long-Term Memory + Retrieval-Augmented Generation)

1.1) Overview

Build an AI assistant that **learns and remembers each user independently over time** by constructing a graph and using **hybrid retrieval** (Graph traversal + optional Vector Search) to answer user questions with context.

Every user has their own "memory space" containing:

- **Facts** (bio, preferences, projects)
- **Entities** (people, places, topics)
- **Events/timelines**
- **Relationships** (connected concepts, dependencies, cause-effect)
- **Confidence / recency / source tracking**

The assistant should **retrieve** the most relevant memories using:

- **Graph queries** (Neo4j preferred, any other graph DB also allowed)
- **Vector similarity search** (Milvus preferred, optional)
- Optional metadata store: **PostgreSQL** (SQL) and/or **MongoDB** (NoSQL)

Finally, it should generate answers using **any accessible/free-tier LLM** (examples: NVIDIA-hosted/free options, Grok/Groq or similar providers if available, or any open/free inference route).

1.2) Core Requirements

A) User-Scoped Memory (Graph)

Create a **graph memory model** per user:

- **Nodes** could include: *User, Entity, Preference, Fact, Message, DocumentChunk, Event, Goal*
- **Edges** could include: *MENTIONS, RELATED_TO, PREFERS, WORKS_ON, HAPPENED_AT, DERIVED_FROM, CONTRADICTS, CONFIRMS*

Must-have: Strict isolation - **User A must never retrieve User B's memory.**

B) Ingestion Pipeline (Memory Writing)

From chat messages and/or uploaded text:

- Extract key information (entities/topics/preferences/events)
- Write to the graph as structured nodes and edges
- Track metadata: timestamp, source type, confidence score, and "last reinforced" time

C) Retrieval + RAG (Memory Reading)

Given a user query:

- Retrieve relevant subgraph via graph query (k-hop neighborhood, path search, entity focus, timeline filters)
- Optionally enrich via vector search (Milvus) over chunks/summaries
- Build a final **context pack** for the LLM: "Top Memories + Evidence + Sources"

D) Answer Generation

Use the retrieved context and query to generate an answer:

- Must show **what memories were used** (brief citations: node IDs/titles/source snippets)
- Must handle "I don't know / not in memory" gracefully

E) Performance Monitoring (MANDATORY)

For every query, teams must display:

- **RAG Retrieval Time** (graph query + vector search + context assembly)
- This time should **exclude LLM response generation time**
- Display format: "Retrieval completed in X ms"

1.3) Suggested Tech Stack (Flexible)

Preferred (not mandatory):

- **Graph DB**: Neo4j
- **Vector DB (optional)**: Milvus
- **SQL (optional)**: PostgreSQL
- **NoSQL (optional)**: MongoDB

LLM:

- Any accessible/free-tier LLM provider (NVIDIA-hosted/free options, Grok/Groq if available, open models via free endpoints, etc.)

1.4) Example Use Cases (Implement at least one)

1. Healthcare Intake Memory (Non-diagnostic): stores user-reported symptoms history, meds, triggers, lifestyle notes (with strict privacy + timestamps); retrieves patterns and summarizes for the next visit (no medical diagnosis).

2. Finance & Spending Coach Memory: remembers budgets, goals, recurring expenses, risk preference; retrieves "your last plan" and updates it based on new constraints.

3. Learning Path Planner: builds a prerequisite graph (topics → prerequisites → resources); for a target skill, generates a personalized roadmap and retrieves what the user already knows.

4. Interview Prep Memory: stores Q/A attempts, weak points, company-specific notes; retrieves last mistakes and drills follow-ups tailored to that topic graph.

5. Travel Planner Memory: remembers preferences (budget, food, pace), past trips, places liked/disliked; retrieves patterns and generates itineraries grounded in stored memory + optional web RAG.

6. Code Review Memory (Engineering Assistant): stores recurring issues, team standards, preferred patterns; retrieves "previous similar PR" explanations and applies consistent review guidance.

7. Academic Advisor Memory: remembers courses taken, grades (optional), interests, research topics, professor interactions; retrieves relevant constraints and suggests next steps.

1.5) Deliverables

1. **GitHub repository** with clean setup instructions
2. **Architecture diagram** (HLD) + brief component explanation
3. **Graph data model** (nodes/edges schema) and sample Cypher queries
4. **API documentation** (endpoints + request/response examples)
5. **Demo** (3–5 min video or live demo):
   - Show memory ingestion
   - Show mindmap/graph visualization
   - Show hybrid retrieval with **retrieval time displayed**
   - Show final grounded answer

1.6) Constraints & Rules

- Must support **multiple users** with isolated memory spaces
- Must include **auditability**: show why a memory was retrieved (path, score, source)
- **Must display RAG retrieval time** (excluding LLM generation) for every query
- Use of vector DB is **optional**, but retrieval must work robustly
- Any UI is acceptable: Streamlit/React/CLI, but should clearly show the mindmap + retrieval evidence

1.7) Stretch Goals (Optional but Impressive)

- **Sub-100ms retrieval time** (graph query + vector search + context assembly)
- **Memory decay / forgetting** (time-based or confidence-based)
- **Contradiction resolution** (multiple facts with confidence + timestamps)
- **Reinforcement learning of memory** (boost nodes used frequently)
- **Hybrid ranker** combining graph score + vector score + recency score
- **Streaming responses** + "memory used" panel like a trace
- **Multi-modal memory** (images, documents, links)

1.8) Suggested APIs (Minimal)

POST /memory/ingest

Body: { user_id, text, source_type }

Response: { success, nodes_created, edges_created }

GET /memory/mindmap

Params: user_id, filters (optional)

Response: { nodes[], edges[], metadata }

POST /chat

Body: { user_id, query }

Response: {

answer,

retrieval_time_ms,

memory_citations: [{ node_id, title, snippet, relevance_score }],

llm_generation_time_ms

}

GET /health

Response: { status, services: { graph_db, vector_db, llm } }

1.9) Getting Started Tips

1. Start small: Begin with a simple graph (User → Fact → Entity)

2. Test with one user first: Get ingestion + retrieval working for a single user

3. Add user isolation: Extend to multiple users with proper data segregation

4. Optimize retrieval: Profile your queries and aim for the sub-100ms stretch goal

5. Visualize early: A simple mindmap viewer helps debugging and demos

Important Note

Teams must implement the memory system from scratch. Direct use of existing memory management libraries like **mem0, LangChain Memory, LlamaIndex Memory, or similar pre-built memory solutions is NOT allowed.** You may use database clients (Neo4j driver, Milvus client), LLM APIs, and general utilities, but the core memory graph architecture, ingestion pipeline, and retrieval logic must be your own implementation.